

1. FCFS Scheduling

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of processes:\n");
    if (scanf("%d", &n) != 1) return 0;

    int at[n], bt[n], ct[n], tat[n], wt[n];
    float total_tat = 0, total_wt = 0;

    for (int i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d:\n", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    for (int i = 0; i < n; i++) {
        if (i == 0)
            ct[i] = at[i] + bt[i];
        else {
            int startTime = (ct[i - 1] > at[i]) ? ct[i - 1] : at[i];
            ct[i] = startTime + bt[i];
        }
    }

    for (int i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        total_tat += tat[i];
        total_wt += wt[i];
    }

    printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);
    printf("Average Waiting Time: %.2f\n", total_wt / n);

    return 0;
}
```

2. Non-Preemptive SJF

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], tat[n], wt[n], done[n];

    for (int i = 0; i < n; i++) {
        printf("Enter AT and BT for P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        done[i] = 0;
    }

    int completed = 0, completionTime = 0;
    float avgtat = 0, avgwt = 0;

    while (completed < n) {
        int idx = -1, minBT = 9999;

        for (int i = 0; i < n; i++) {
            if (at[i] <= completionTime && !done[i] && bt[i] < minBT) {

```

```

        minBT = bt[i];
        idx = i;
    }
}

if (idx != -1) {
    if (completionTime < at[idx])
        completionTime = at[idx];

    completionTime += bt[idx];

    tat[idx] = completionTime - at[idx];
    wt[idx] = tat[idx] - bt[idx];

    avgtat += tat[idx];
    avgwt += wt[idx];

    done[idx] = 1;
    completed++;
} else {
    completionTime++;
}
}

printf("\nAverage TAT: %.2f\n", avgtat / n);
printf("Average WT: %.2f\n", avgwt / n);

return 0;
}

```

3. Round Robin

```

#include <stdio.h>

int main() {
    int n, tq;
    printf("Enter number of processes:\n");
    scanf("%d", &n);

    printf("Enter Time Quantum:\n");
    scanf("%d", &tq);

    int at[n], bt[n], rem[n], tat[n], wt[n];
    int completed = 0, time = 0;
    float avgtat = 0, avgwt = 0;

    for (int i = 0; i < n; i++) {
        printf("Enter AT and BT for P%d:\n", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        rem[i] = bt[i];
    }

    while (completed < n) {
        int idle = 1;

        for (int i = 0; i < n; i++) {
            if (at[i] <= time && rem[i] > 0) {
                idle = 0;

                if (rem[i] > tq) {
                    time += tq;
                    rem[i] -= tq;
                } else {
                    time += rem[i];
                    tat[i] = time - at[i];
                }
            }
        }

        if (idle == 1) {
            printf("No process to execute\n");
            continue;
        }

        completed++;
    }

    printf("\nAverage TAT: %.2f\n", avgtat / n);
    printf("Average WT: %.2f\n", avgwt / n);

    return 0;
}

```

```

        wt[i] = tat[i] - bt[i];
        rem[i] = 0;
        completed++;
    }
}

if (idle) time++;
}

for (int i = 0; i < n; i++) {
    avgtat += tat[i];
    avgwt += wt[i];
}

printf("\nAverage TAT: %.2f\n", avgtat / n);
printf("Average WT: %.2f\n", avgwt / n);

return 0;
}

```

4. Priority Scheduling

```

#include <stdio.h>

int main() {
    int n, time = 0, completed = 0;

    printf("Enter number of processes:\n");
    scanf("%d", &n);

    int at[n], bt[n], priority[n], done[n];
    int ct[n], tat[n], wt[n];
    float avgtat = 0, avgwt = 0;

    for (int i = 0; i < n; i++) {
        printf("Enter AT, BT, Priority for P%d:\n", i + 1);
        scanf("%d %d %d", &at[i], &bt[i], &priority[i]);
        done[i] = 0;
    }

    while (completed < n) {
        int idx = -1, highPriority = 9999;

        for (int i = 0; i < n; i++) {
            if (at[i] <= time && !done[i] && priority[i] < highPriority) {
                highPriority = priority[i];
                idx = i;
            }
        }

        if (idx != -1) {
            time += bt[idx];
            ct[idx] = time;

            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];

            avgtat += tat[idx];
            avgwt += wt[idx];

            done[idx] = 1;
            completed++;
        } else {
            time++;
        }
    }
}

```

```

    }
}

printf("\nAverage TAT: %.2f\n", avgtat / n);
printf("Average WT: %.2f\n", avgwt / n);

return 0;
}

```

5. FIFO Page Replacement

```

#include <stdio.h>

int main() {
    int pages, frames;

    printf("Enter number of pages:\n");
    scanf("%d", &pages);

    int ref[pages];
    printf("Enter reference string:\n");
    for (int i = 0; i < pages; i++)
        scanf("%d", &ref[i]);

    printf("Enter number of frames:\n");
    scanf("%d", &frames);

    int frame[frames];
    for (int i = 0; i < frames; i++)
        frame[i] = -1;

    int index = 0, faults = 0, hits = 0;

    for (int i = 0; i < pages; i++) {
        int found = 0;

        for (int j = 0; j < frames; j++) {
            if (frame[j] == ref[i]) {
                found = 1;
                hits++;
                break;
            }
        }

        if (!found) {
            frame[index] = ref[i];
            index = (index + 1) % frames;
            faults++;
        }
    }

    printf("\nPage Faults: %d\n", faults);
    printf("Page Hits: %d\n", hits);

    return 0;
}

```

6. LRU Page Replacement

```

#include <stdio.h>

int main() {

```

```

int n, frames;

printf("Enter number of pages:\n");
scanf("%d", &n);

int pages[n];
printf("Enter reference string:\n");
for (int i = 0; i < n; i++)
    scanf("%d", &pages[i]);

printf("Enter number of frames:\n");
scanf("%d", &frames);

int frame[frames], time[frames];

for (int i = 0; i < frames; i++) {
    frame[i] = -1;
    time[i] = 0;
}

int faults = 0, hits = 0, counter = 0;

for (int i = 0; i < n; i++) {
    int found = 0;

    for (int j = 0; j < frames; j++) {
        if (frame[j] == pages[i]) {
            found = 1;
            hits++;
            counter++;
            time[j] = counter;
            break;
        }
    }

    if (!found) {
        int lru = 0;

        for (int j = 1; j < frames; j++) {
            if (time[j] < time[lru])
                lru = j;
        }

        counter++;
        frame[lru] = pages[i];
        time[lru] = counter;
        faults++;
    }
}

printf("\nPage Faults: %d\n", faults);
printf("Page Hits: %d\n", hits);

return 0;
}

```

1. IPC – Shared Memory

sender.c

```

#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

```

```

#define SharedMemSize 50

int main() {
    int shmid;
    key_t key = 5677;
    char *shared_memory;

    // Create shared memory
    if ((shmid = shmget(key, SharedMemSize, IPC_CREAT | 0666)) < 0) {
        perror("shmget");
        exit(1);
    }

    // Attach
    if ((shared_memory = shmat(shmid, NULL, 0)) == (char *)-1) {
        perror("shmat");
        exit(1);
    }

    sprintf(shared_memory, "Welcome to Shared Memory");

    sleep(2);
    return 0;
}

```

receiver.c

```

#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <stdlib.h>

#define SharedMemSize 50

int main() {
    int shmid;
    key_t key = 5677;
    char *shared_memory;

    if ((shmid = shmget(key, SharedMemSize, 0666)) < 0) {
        perror("shmget");
        exit(1);
    }

    if ((shared_memory = shmat(shmid, NULL, 0)) == (char *)-1) {
        perror("shmat");
        exit(1);
    }

    printf("Message Received: %s\n", shared_memory);

    return 0;
}

```

2. Producer–Consumer (Semaphores)

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

```

```

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t mutex, empty, full;

void* producer(void* arg) {
    for (int i = 0; i < 10; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);

        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&full);

        sleep(1);
    }
    return NULL;
}

void* consumer(void* arg) {
    for (int i = 0; i < 10; i++) {
        sem_wait(&full);
        sem_wait(&mutex);

        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&empty);

        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t p, c;

    sem_init(&mutex, 0, 1);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&mutex);
    sem_destroy(&empty);
    sem_destroy(&full);

    return 0;
}

```

3. Banker's Algorithm

```

#include <stdio.h>

int main() {

```

```

int n, r;

printf("Enter number of processes: ");
scanf("%d", &n);

printf("Enter number of resource types: ");
scanf("%d", &r);

int alloc[n][r], max[n][r], need[n][r], avail[r];
int finish[n], safe_sequence[n];

// Allocation
printf("Enter Allocation Matrix:\n");
for (int i = 0; i < n; i++)
    for (int j = 0; j < r; j++)
        scanf("%d", &alloc[i][j]);

// Max
printf("Enter Max Matrix:\n");
for (int i = 0; i < n; i++)
    for (int j = 0; j < r; j++)
        scanf("%d", &max[i][j]);

// Available
printf("Enter Available Resources:\n");
for (int i = 0; i < r; i++)
    scanf("%d", &avail[i]);

// Initialize
for (int i = 0; i < n; i++)
    finish[i] = 0;

// Need = Max - Alloc
for (int i = 0; i < n; i++)
    for (int j = 0; j < r; j++)
        need[i][j] = max[i][j] - alloc[i][j];

int count = 0;

while (count < n) {
    int safeFound = 0;

    for (int i = 0; i < n; i++) {
        if (!finish[i]) {
            int safe = 1;

            for (int j = 0; j < r; j++) {
                if (need[i][j] > avail[j]) {
                    safe = 0;
                    break;
                }
            }

            if (safe) {
                for (int k = 0; k < r; k++)
                    avail[k] += alloc[i][k];

                safe_sequence[count++] = i;
                finish[i] = 1;
                safeFound = 1;
            }
        }
    }

    if (!safeFound) {
        printf("System is in unsafe state!\n");
        return 0;
    }
}

```



```

    }
}

printf("Safe sequence: ");
for (int i = 0; i < n; i++)
    printf("P%d ", safe_sequence[i]);

return 0;
}

```

4. First Fit

```

#include <stdio.h>

int main() {
    int m, n;

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int blockSize[m], remaining[m];

    printf("Enter block sizes:\n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);
        remaining[i] = blockSize[i];
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n);

    int processSize[n];

    printf("Enter process sizes:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &processSize[i]);

    for (int i = 0; i < n; i++) {
        int allocated = 0;

        for (int j = 0; j < m; j++) {
            if (remaining[j] >= processSize[i]) {
                int fragment = remaining[j] - processSize[i];

                printf("P%d -> Block %d (Fragment %d)\n",
                    i + 1, j + 1, fragment);

                remaining[j] -= processSize[i];
                allocated = 1;
                break;
            }
        }

        if (!allocated)
            printf("P%d not allocated\n", i + 1);
    }

    return 0;
}

```

5. Best Fit

```

#include <stdio.h>

int main() {
    int m, n;

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int blockSize[m], remaining[m];

    printf("Enter block sizes:\n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);
        remaining[i] = blockSize[i];
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n);

    int processSize[n];

    printf("Enter process sizes:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &processSize[i]);

    for (int i = 0; i < n; i++) {
        int bestIndex = -1;

        for (int j = 0; j < m; j++) {
            if (remaining[j] >= processSize[i]) {
                if (bestIndex == -1 || remaining[j] < remaining[bestIndex]) {
                    bestIndex = j;
                }
            }
        }

        if (bestIndex != -1) {
            int fragment = remaining[bestIndex] - processSize[i];

            printf("P%d -> Block %d (Fragment %d)\n",
                i + 1, bestIndex + 1, fragment);

            remaining[bestIndex] -= processSize[i];
        } else {
            printf("P%d not allocated\n", i + 1);
        }
    }

    return 0;
}

```

Linux Commands

1. `ls` – List Files and Directories

`ls`

2. `pwd` – Print Working Directory

`pwd`

3. `mkdir` – Create Directory

```
mkdir myfolder
```

4. `cd` – Change Directory

```
cd myfolder
```

5. `touch` – Create File

```
touch file.txt
```

6. `cp` – Copy File

```
cp file1.txt file2.txt
```

7. `mv` – Move/Rename File

```
mv old.txt new.txt
```

8. `rm` – Remove File

```
rm file.txt
```

9. `cat` – Display File Content

```
cat file.txt
```

10. `grep` – Search Pattern

```
grep "hello" file.txt
```

Text Editors

1. Using `vi` Editor

```
vi file.txt
```

Important Commands

Command	Purpose
i	Insert mode
Esc	Exit insert mode
:w	Save
:q	Quit
:wq	Save and quit

2. Using nano

```
nano file.txt
```

Shortcut Purpose

Ctrl + O Save

Ctrl + X Exit

Shell Script

Addition of Two Numbers

```
#!/bin/bash

echo "Enter first number:"
read a

echo "Enter second number:"
read b

sum=$((a + b))

echo "Sum = $sum"
```

Factorial Program

```
#!/bin/bash

echo "Enter a number:"
read n

fact=1

for ((i=1; i<=n; i++))
do
    fact=$((fact * i))
done

echo "Factorial = $fact"
```

Even or Odd

```
#!/bin/bash

echo "Enter a number:"
read n

if [ $((n % 2)) -eq 0 ]
then
    echo "Even"
else
    echo "Odd"
fi
```

AWK Script

Print File Contents

```
awk '{print}' file.txt
```

Print First Column

```
awk '{print $1}' file.txt
```

Print Lines Containing Word

```
awk '/hello/ {print}' file.txt
```

Sum of Numbers

```
awk '{sum += $1} END {print sum}' file.txt
```

System Calls

1. `fork()` – Create Process

```
#include <stdio.h>
#include <unistd.h>

int main() {
    fork();

    printf("Process Created\n");

    return 0;
}
```

2. `getpid()` – Get Process ID

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Process ID: %d\n", getpid());

    return 0;
}
```

3. `wait()` – Parent Waits for Child

```
#include <stdio.h>
#include <sys/types.h>
```

```
#include <sys/wait.h>
#include <unistd.h>

int main() {
    fork();

    wait(NULL);

    printf("Completed\n");

    return 0;
}
```

4. `exec()` – Execute New Program

```
#include <stdio.h>
#include <unistd.h>

int main() {
    execl("/bin/ls", "ls", NULL);

    return 0;
}
```